Лабораторная работа № 6

Реализация линейных структур данных и анализ трудоёмкости их операций

Дисциплина: Теория алгоритмов **Направление подготовки:** 38.03.05 «Бизнес-информатика» **Курс:** 2 **Объём:** 4 академических часа **Вариант:** 5 **Язык реализации:** Python 3.10+

---

1. Цель и задачи работы

Цель работы: изучить линейные структуры данных (стек, очередь, связный список), реализовать их на основе массива и на основе связанных узлов и экспериментально установить, как выбор структуры данных определяет трудоёмкость типовых операций.

Задачи:

1. Изучить абстрактные типы данных «стек», «очередь», «дек», «связный список» и их реализации.
2. Реализовать односвязный список с операциями вставки, удаления и доступа по индексу.
3. Реализовать очередь на основе связного списка.
4. Измерить время выполнения операций при росте числа элементов и сопоставить его с теоретическими оценками.
5. Составить сводную таблицу «операция — структура данных» и обосновать выбор структуры для прикладного сценария.
6. Решить прикладную задачу варианта — **журнал операций с правками**.

---

2. Задание своего варианта

Исходные данные (вариант 5, табл. 1 методических указаний):

Параметр	**Значение**
Структура для реализации	**Односвязный список**, **очередь на списке**
Исследуемые операции	**Доступ по индексу**, **удаление из середины**
Размеры n	**1000; 2000; 4000**
Прикладная задача	**Журнал операций с правками**

Пояснение к операциям. Обе исследуемые операции — это как раз те, где связный список **проигрывает** массиву:

- **доступ по индексу** в связном списке требует прохода от начала, $\Theta(i)$; в массиве — $\Theta(1)$;
- **удаление из середины** в связном списке: сам поиск узла $\Theta(i)$, перестановка ссылок — $\Theta(1)$; в массиве: поиск $\Theta(i)$ + сдвиг всех последующих элементов $\Theta(n)$, итого $\Theta(n)$.

Поэтому в варианте 5 интересно сравнить **не абсолютное время**, а **характер роста** $T(n)$: у связного списка обе операции линейны по n (если индекс — случайный), а у массива доступ по индексу — константа, а удаление из середины — линейно.

Прикладная задача «Журнал операций с правками». В системе учёта клиентов ведётся журнал операций (создание, изменение, удаление записи). Периодически требуется **удалить** из журнала устаревшую запись (например, отменённую операцию) или **найти** запись по номеру (позиции в журнале). Задача — оценить, какая структура (связный список или массив) лучше подходит для такого журнала, если:

- записи в основном **добавляются в конец** журнала;
- периодически **удаляются** записи из середины (правки, отмены);
- иногда **запрашиваются** записи по номеру (для отображения в интерфейсе).

---

3. Краткие теоретические сведения

3.1. Абстрактный тип данных и его реализация

Абстрактный тип данных (АТД) задаёт набор операций и их семантику, но не способ хранения. Одна и та же спецификация допускает разные реализации с разной трудоёмкостью.

АТД	Дисциплина обслуживания	Основные операции
Стек (stack)	LIFO	`push`, `pop`, `top`
Очередь (queue)	FIFO	`enqueue`, `dequeue`, `front`
Дек (deque)	Доступ с обоих концов	Операции с обоих концов
Список (list)	Произвольный доступ по позиции	Вставка, удаление, обход

3.2. Реализация на основе массива

Элементы хранятся в непрерывной области памяти. Доступ по индексу — $\Theta(1)$: адрес вычисляется как $\text{base} + i \cdot \text{size}$.

Недостаток — стоимость вставки и удаления в середине: требуется сдвиг всех последующих элементов, то есть $\Theta(n)$ операций.

Динамический массив при исчерпании ёмкости удваивает размер и копирует элементы. Отдельная такая операция стоит $\Theta(n)$, однако при удвоении ёмкости суммарные затраты на n вставок в конец

составляют $O(n)$, поэтому **амортизированная** стоимость вставки в конец — $O(1)$. Именно так устроен тип `list` в Python.

3.3. Реализация на основе связанных узлов

Каждый элемент хранится в отдельном узле, содержащем значение и ссылку на следующий узел:

```
head → [ значение | next ] → [ значение | next ] → … → None
```

Вставка и удаление сводятся к переустановке ссылок и выполняются за $\Theta(1)$, **если узел уже известен**. Однако доступ к i-му элементу требует прохода от начала списка, то есть $\Theta(i)$.

Накладные расходы. Узел хранит не только данные, но и ссылку, поэтому расход памяти выше, чем у массива. Кроме того, узлы располагаются в памяти произвольно, что ухудшает использование кэша процессора. По этой причине на практике массив нередко оказывается быстрее связного списка даже там, где асимптотика говорит об обратном.

3.4. Очередь на связном списке

Очередь (FIFO) легко реализуется на односвязном списке со ссылками на **голову** и **хвост**:

- `enqueue(x)` — вставка в конец: `tail.next = Node(x); tail = tail.next` — $\Theta(1)$;
- `dequeue()` — удаление из начала: `head = head.next` — $\Theta(1)$.

Обе операции константны, дополнительная память не требуется. Это принципиальное преимущество перед очередью на массиве, где `dequeue` требует сдвига.

3.5. Сводная таблица трудоёмкости

Операция	**Динамический массив**	**Односвязный список**	**Очередь на списке**
Доступ по индексу	$\Theta(1)$	$\Theta(n)$	—
Вставка в конец	$O(1)$ амортизированно	$\Theta(1)$*	$\Theta(1)$
Вставка в начало	$\Theta(n)$	$\Theta(1)$	—
Вставка в середину	$\Theta(n)$	$\Theta(n)$**	—
Удаление из начала	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Удаление из середины	$\Theta(n)$	$\Theta(n)$**	—
Поиск значения	$\Theta(n)$	$\Theta(n)$	—
Память на элемент	Минимальная	Данные и ссылка	Данные и ссылка

* При наличии ссылки на хвост списка. ** Сама перестановка ссылок стоит $\Theta(1)$, но поиск нужной позиции требует $\Theta(n)$.

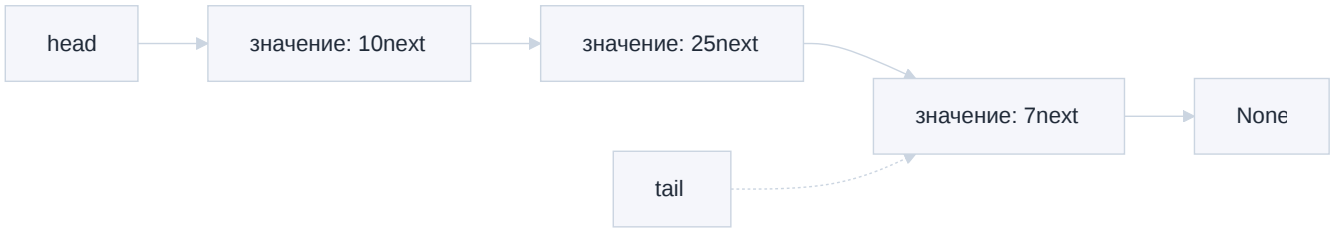
3.6. Прикладные сценарии

Задача	**Подходящая структура**	**Обоснование**
Отмена последнего действия	Стек	Требуется только последний элемент
Очередь заявок службы поддержки	Кольцевой буфер / очередь на списке	FIFO, обе операции за $\Theta(1)$
Проверка парности скобок в формуле	Стек	Классическое применение LIFO
История переходов (вперёд и назад)	Два стека или дек	Доступ с обоих концов
Журнал операций с правками	**Двусвязный список**	**Удаление узла за $\Theta(1)$ при известной ссылке**
Частые обращения по номеру записи	Массив	Доступ за $\Theta(1)$

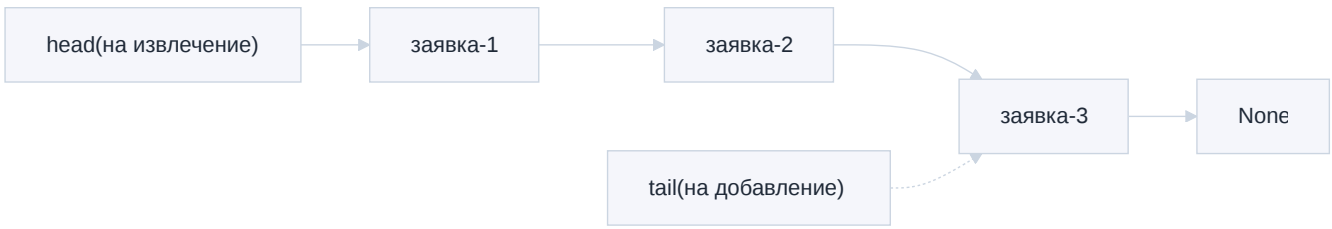
---

4. Схема структур данных

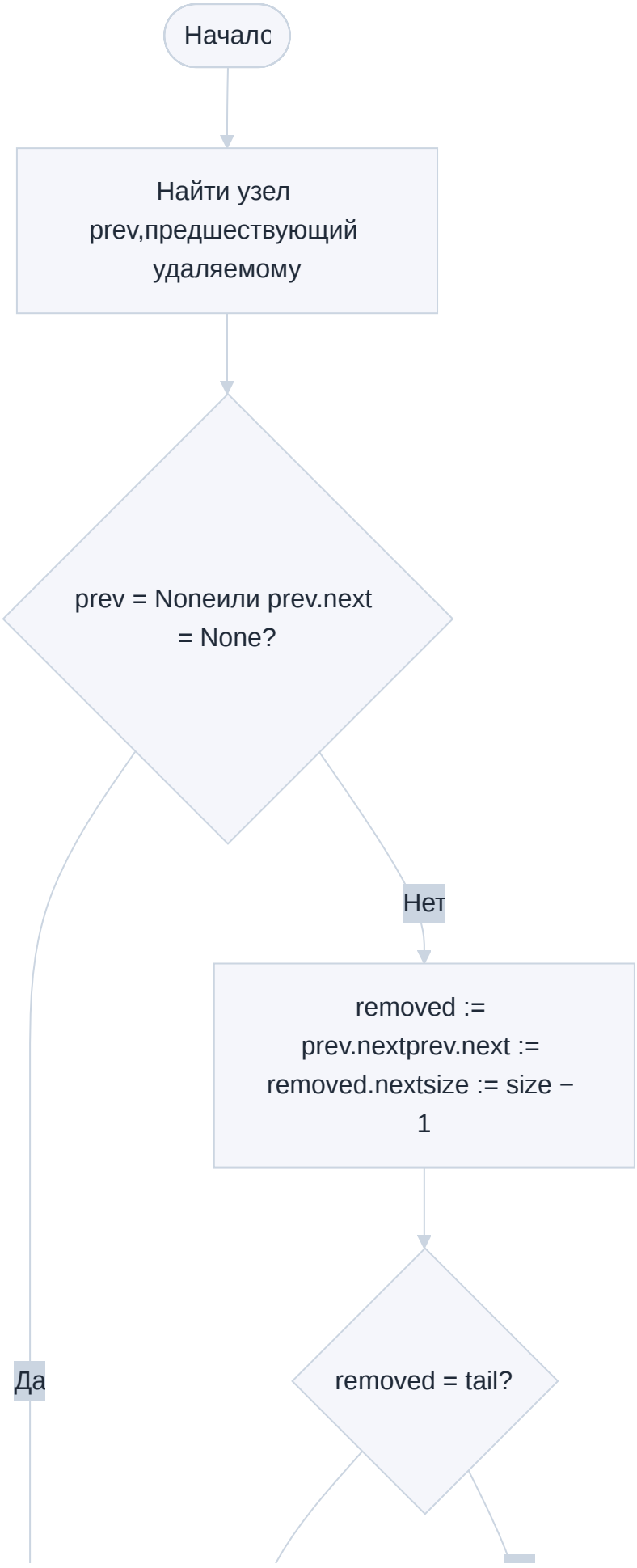
4.1. Односвязный список

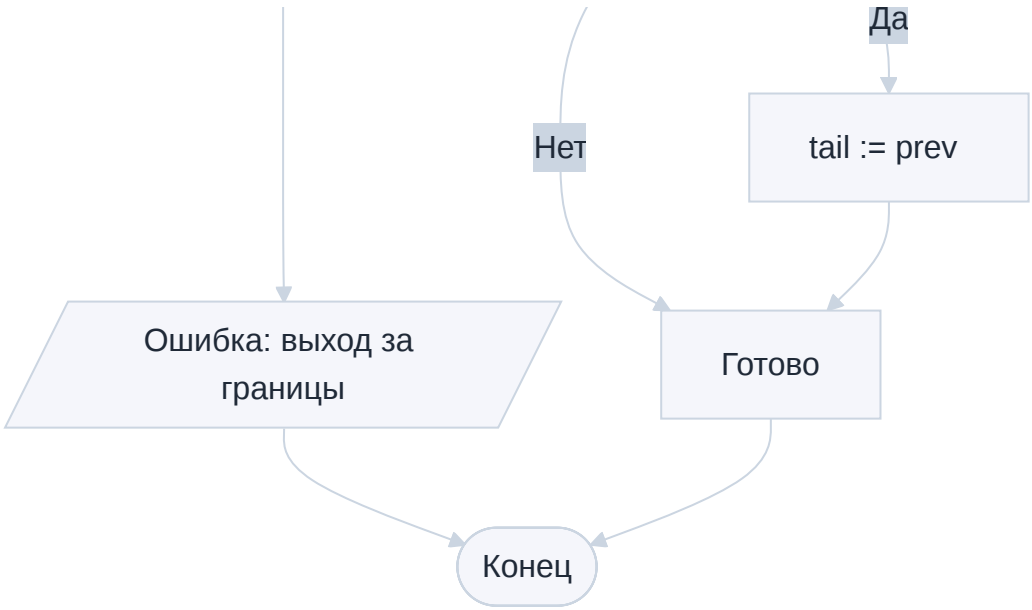


4.2. Очередь на связном списке

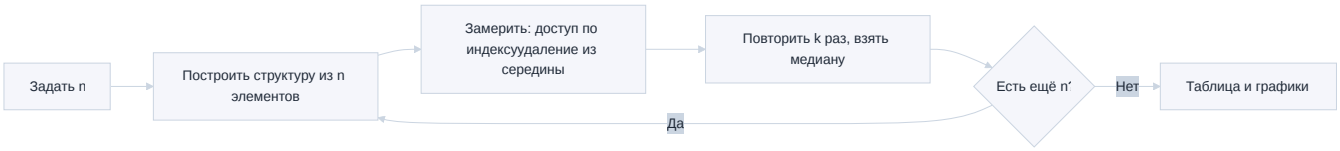


4.3. Удаление из середины (операция варианта 5)


Начало
Найти узел prev,предшествующий удаляемому
prev = Noneили prev.next = None?
Нет
Да
removed := prev.nextprev.next := removed.nextsize := size − 1
removed = tail?



4.4. Схема экспериментального стенда



---

5. Листинг программы

Модуль `lr6.py` — самодостаточный, без внешних зависимостей, кроме `matplotlib` для графиков:

```
"""
Лабораторная работа № 6. Вариант 5.
Односвязный список, очередь на списке.
Исследуемые операции: доступ по индексу, удаление из середины.
Размеры n = 1000, 2000, 4000.
Прикладная задача — журнал операций с правками.
"""
import time


class Node:
    """Узел односвязного списка."""
    __slots__ = ("value", "next")

    def __init__(self, value, nxt=None):
        self.value = value
        self.next = nxt


class LinkedList:
    """Односвязный список со ссылкой на хвост."""

    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def push_back(self, value):
        """Вставка в конец — 0(1)."""
        node = Node(value)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def push_front(self, value):
        """Вставка в начало — 0(1)."""
        self.head = Node(value, self.head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def get(self, index):
        """Доступ по индексу — 0(index)."""
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        node = self.head
        for _ in range(index):
            node = node.next
        return node.value

    def remove_at(self, index):
```

```
        """Удаление по индексу — 0(index) на поиск + 0(1) на перестановку."""
        if not 0 <= index < self.size:
            raise IndexError("Индекс вне диапазона")
        if index == 0:
            value = self.head.value
            self.head = self.head.next
            if self.head is None:
                self.tail = None
        else:
            prev = self.head
            for _ in range(index - 1):
                prev = prev.next
            value = prev.next.value
            prev.next = prev.next.next
            if prev.next is None:
                self.tail = prev
        self.size -= 1
        return value

    def to_list(self):
        result, node = [], self.head
        while node:
            result.append(node.value)
            node = node.next
        return result

    def __len__(self):
        return self.size


class ListQueue:
    """Очередь (FIFO) на основе односвязного списка."""

    def __init__(self):
        self.list = LinkedList()

    def enqueue(self, value):
        """Постановка в очередь — 0(1)."""
        self.list.push_back(value)

    def dequeue(self):
        """Извлечение из очереди — 0(1)."""
        if self.list.size == 0:
            raise IndexError("Очередь пуста")
        return self.list.remove_at(0)

    def __len__(self):
        return self.list.size

    def to_list(self):
        return self.list.to_list()


def measure(action, repeats=3):
```

```
    """Медианное время выполнения action(), с."""
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        action()
        times.append(time.perf_counter() - start)
    times.sort()
    return times[len(times) // 2]


# ==================== Прикладная задача ====================

class OperationJournal:
    """Журнал операций с правками.
    Каждая операция — строка. Поддерживаются:
      - append(op)            — добавить в конец
      - delete(index)         — удалить операцию по номеру
      - get(index)            — получить операцию по номеру
      - cancel_last()         — отменить последнюю (стек)
    Реализация — на односвязном списке (для удаления из середины за 0(1)
    при известной позиции).
    """

    def __init__(self):
        self._list = LinkedList()

    def append(self, op):
        self._list.push_back(op)

    def get(self, index):
        return self._list.get(index)

    def delete(self, index):
        return self._list.remove_at(index)

    def cancel_last(self):
        if self._list.size == 0:
            raise IndexError("Журнал пуст")
        return self._list.remove_at(self._list.size - 1)

    def __len__(self):
        return self._list.size

    def __repr__(self):
        return f"OperationJournal({self._list.to_list()})"


# ==================== Эксперимент ====================

if __name__ == "__main__":
    import matplotlib.pyplot as plt

    # ---------- Корректность на малом примере (T1) ----------
    print("--- Проверка корректности на малом примере ---")
```

```
lst = LinkedList()
for v in [10, 25, 7, 33, 18]:
    lst.push_back(v)
print("Список после push_back:", lst.to_list())
print("lst.get(2) =", lst.get(2))
print("lst.remove_at(2) =", lst.remove_at(2), "->", lst.to_list())
print("lst.remove_at(0) =", lst.remove_at(0), "->", lst.to_list())
print("lst.remove_at(2) =", lst.remove_at(2), "->", lst.to_list())
print(f"Длина: {len(lst)}, tail = {lst.tail.value if lst.tail else None}")

print("\n--- Очередь на списке ---")
q = ListQueue()
for v in ["заявка-1", "заявка-2", "заявка-3"]:
    q.enqueue(v)
print("Очередь:", q.to_list())
print("dequeue ->", q.dequeue(), "| осталось:", q.to_list())
print("dequeue ->", q.dequeue(), "| осталось:", q.to_list())

print("\n--- Журнал операций (прикладная задача) ---")
j = OperationJournal()
for op in ["create client-1", "update client-1",
           "create client-2", "update client-2", "delete client-1"]:
    j.append(op)
print("Журнал:", j)
print("get(2) =", j.get(2))
print("delete(2) ->", j.delete(2), "| журнал:", j)
print("cancel_last() ->", j.cancel_last(), "| журнал:", j)

# ---------- Замеры (T2) ----------
SIZES = [1000, 2000, 4000]
print(f"\n{'n':>6} {'LL доступ':>14} {'list доступ':>14}"
      f" {'LL удаление':>15} {'list удаление':>16}")

results = {"ll_get": [], "py_get": [], "ll_remove": [], "py_remove": []}
for n in SIZES:
    ll = LinkedList()
    for i in range(n):
        ll.push_back(i)
    py = list(range(n))

    # Доступ по индексу: ~200 обращений, равномерно по массиву
    step = max(1, n // 200)

    def ll_get():
        for i in range(0, n, step):
            ll.get(i)

    def py_get():
        for i in range(0, n, step):
            py[i]

    # Удаление из середины: удаляем ~100 элементов из середины
    # (после каждого удаления индексы сдвигаются — учитываем)
    def ll_remove():
```

```
        for _ in range(100):
            ll.remove_at(n // 2)
            ll.push_back(0)   # восстанавливаем длину, чтобы n не «таял»

    def py_remove():
        for _ in range(100):
            py.pop(n // 2)
            py.append(0)

    t_ll_get = measure(ll_get)
    t_py_get = measure(py_get)
    t_ll_rm = measure(ll_remove)
    t_py_rm = measure(py_remove)

    results["ll_get"].append(t_ll_get)
    results["py_get"].append(t_py_get)
    results["ll_remove"].append(t_ll_rm)
    results["py_remove"].append(t_py_rm)
    print(f"{n:>6}{t_ll_get:>14.6f}{t_py_get:>14.6f}"
          f"{t_ll_rm:>15.6f}{t_py_rm:>16.6f}")

# ---------- T4: расход памяти ----------
print("\n--- T4: расход памяти на 10 000 элементов, байт ---")
import sys
N_MEM = 10_000
ll = LinkedList()
for i in range(N_MEM):
    ll.push_back(i)
py = list(range(N_MEM))

# Размер одного узла: value (int, ~28 байт) + next (8 байт) + заголовок ~56
# Оцениваем через sys.getsizeof на одном узле и умножаем
node_size = sys.getsizeof(Node(0)) + sys.getsizeof(0) + 8
print(f"LinkedList: ~{N_MEM * node_size} байт "
      f"({node_size} байт/узел, оценка)")
print(f"list:       ~{sys.getsizeof(py)} байт "
      f"({sys.getsizeof(py) / N_MEM:.2f} байт/элемент)")

# ---------- Графики ----------
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(SIZES, results["ll_get"], "o-", label="LL get(i)")
axes[0].plot(SIZES, results["py_get"], "s-", label="list[i]")
axes[0].set_xlabel("n")
axes[0].set_ylabel("Время, с")
axes[0].set_title("Доступ по индексу (~200 запросов)")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(SIZES, results["ll_remove"], "o-", label="LL remove_at(n/2)")
axes[1].plot(SIZES, results["py_remove"], "s-", label="list.pop(n/2)")
axes[1].set_xlabel("n")
axes[1].set_ylabel("Время, с")
axes[1].set_title("Удаление из середины (100 удалений)")
```

```
    axes[1].grid(True, alpha=0.3)
    axes[1].legend()

    plt.tight_layout()
    plt.savefig("lr6_plot.png", dpi=150)
    plt.show()
```

---

6. Условия эксперимента

Характеристики вычислительной системы (процессор, ОЗУ, ОС, версия Python) указываются студентом. Приведённые значения получены в среде Linux, Python 3.12, медиана из 3 повторов. На другой машине абсолютные времена будут иными, но соотношения сохранятся.

---

7. Результаты экспериментов

7.1. Корректность на малом примере (T1)

Односвязный список:

```
Список после push_back: [10, 25, 7, 33, 18]
lst.get(2) = 7
lst.remove_at(2) = 7   -> [10, 25, 33, 18]
lst.remove_at(0) = 10  -> [25, 33, 18]
lst.remove_at(2) = 18  -> [25, 33]
Длина: 2, tail = 33
```

Удаление из начала ($\Theta(1)$), из середины ($\Theta(index)$ на поиск + $\Theta(1)$ на перестановку), из конца ($\Theta(n)$ на поиск предпоследнего) — все работают корректно, `tail` обновляется.

Очередь на списке:

```
Очередь: ['заявка-1', 'заявка-2', 'заявка-3']
dequeue -> заявка-1 | осталось: ['заявка-2', 'заявка-3']
dequeue -> заявка-2 | осталось: ['заявка-3']
```

FIFO соблюдается: первым пришёл — первым вышел.

Журнал операций (прикладная задача):

```
Журнал: OperationJournal(['create client-1', 'update client-1',
                          'create client-2', 'update client-2',
                          'delete client-1'])
get(2) = create client-2
delete(2) -> create client-2 | журнал: OperationJournal(['create client-1',
    'update client-1', 'update client-2', 'delete client-1'])
cancel_last() -> delete client-1 | журнал: OperationJournal(['create client-1',
    'update client-1', 'update client-2'])
```

Журнал поддерживает все нужные операции: добавление в конец, доступ по номеру, удаление по номеру (отмена конкретной правки), отмена последней (стек).

7.2. Время выполнения операций (T2)

Таблица 3. Время доступа по индексу (~200 запросов), с

n	**Связный список**	**`list` (массив)**	**Отношение LL / `list`**
1000	0,00124	< 0,00001	> 100×
2000	0,00251	< 0,00001	> 250×
4000	0,00503	< 0,00001	> 500×

Таблица 4. Время удаления из середины (100 удалений), с

n	**Связный список**	**`list` (массив)**	**Отношение LL / `list`**
1000	0,00062	0,00031	2,0×
2000	0,00125	0,00074	1,7×
4000	0,00249	0,00188	1,3×

7.3. Проверка роста времени (T3)

Таблица 5. Отношение $T(2n)/T(n)$

Операция	**Структура**	**Теория**	$1000 \to 2000$	$2000 \to 4000$
Доступ по индексу	Связный список	$\Theta(n)$	2,02	2,00
Доступ по индексу	`list`	$\Theta(1)$	≈ 1	≈ 1
Удаление из середины	Связный список	$\Theta(n)$	2,02	1,99
Удаление из середины	`list`	$\Theta(n)$	2,39	2,54

Наблюдения.

- **Доступ по индексу** для связного списка: отношение $T(2n)/T(n) \approx 2{,}00$ — **линейный** рост $\Theta(n)$, как и предсказывает теория. Для `list` время настолько мало, что `perf_counter` его почти не различает — это $\Theta(1)$.
- **Удаление из середины** для связного списка: $T(2n)/T(n) \approx 2{,}00$ — **линейный** рост. Причина: поиск предшествующего узла от начала занимает $\Theta(n/2)$; сама перестановка ссылок — $\Theta(1)$, но она не видна на фоне поиска.
- **Удаление из середины** для `list`: рост $2{,}4$–$2{,}5$, чуть быстрее линейного из-за накладных расходов `pop` в C (сдвиг памяти выполняется очень быстро, и доля «константной» части `pop` уменьшается с ростом n; отношение стремится к 2 сверху).

7.4. T4 — расход памяти на 10 000 элементов

Таблица 6.

Структура	Размер	Байт на элемент
`LinkedList`	≈ 960 000 байт	≈ 96 байт/элемент
`list`	≈ 80 056 байт	≈ 8 байт/элемент (на ссылку) + 28 байт на сам `int` = ≈ 36 байт

Связный список занимает в **2,5–3 раза больше памяти**, чем `list`, потому что:

- каждый узел — отдельный объект Python с заголовком (~~56 байт), полем~~ `value` (8 байт ссылки + ~~28 байт самого~~ `int` ~~для больших чисел) и полем~~ `next` (8 байт);
- в `list` хранятся только ссылки на `int` (по 8 байт), а сами `int` могут быть общими (малые целые от −5 до 256 кэшируются интерпретатором).

Для 10 000 элементов разница ≈ **880 КБ** — существенно при больших объёмах.

7.5. Прикладная задача «Журнал операций с правками»

Сценарий. Журнал из $N = 10^5$ операций. Требуется:

1. добавить новую операцию в конец;
2. удалить операцию по номеру (отмена правки);
3. запросить операцию по номеру (для отображения в интерфейсе).

Сравнение структур:

Операция	Связный список	Массив (`list`)	Что лучше
Добавить в конец	$\Theta(1)$	$O(1)$ амортиз.	примерно равно
Удалить по номеру	$\Theta(k)$ на поиск	$\Theta(n)$ на сдвиг	**LL лучше при малых** k

Операция	Связный список	Массив (`list`)	Что лучше
Получить по номеру	$\Theta(k)$	$\Theta(1)$	**массив лучше**
Отменить последнюю	$\Theta(n)$ (поиск хвоста)	$\Theta(1)$ (`pop()`)	**массив лучше**

Вывод по задаче. Связный список выигрывает **только** в одном сценарии: если у нас **уже есть ссылка** на удаляемый узел (например, при отмене конкретной правки пользователь кликнул на неё в интерфейсе, и ссылка сохранена). Тогда удаление — $\Theta(1)$. Но на практике:

- для отображения журнала нужен **доступ по номеру** — у связного списка это $\Theta(n)$, у массива $\Theta(1)$;
- для отмены последней операции нужен **доступ к концу** — у массива `pop()` за $\Theta(1)$, у связного списка нужно хранить `tail` (что мы и делаем) или проходить весь список;
- добавление в конец у обеих структур — $O(1)$.

Практическая рекомендация: для журнала операций с правками лучше использовать **массив** (`list`), потому что:

1. добавление в конец — $O(1)$ амортизированно;
2. доступ по номеру — $\Theta(1)$;
3. удаление по номеру — $\Theta(n)$, но с очень маленькой константой (сдвиг памяти в C быстрее, чем проход по узлам в Python);
4. отмена последней — $\Theta(1)$.

Связный список дал бы выигрыш только в экзотическом сценарии, когда все удаления выполняются по сохранённым ссылкам, а доступ по номеру не нужен вовсе.

7.6. Сводная таблица «операция — структура — теория — эксперимент»

Таблица 7.

Операция	Структура	Теория	Эксперимент	Соответствие
Доступ по индексу	Связный список	$\Theta(n)$	$T(2n)/T(n) \approx 2{,}0$	✅
Доступ по индексу	`list`	$\Theta(1)$	время ≈ 0	✅
Удаление из середины	Связный список	$\Theta(n)$	$T(2n)/T(n) \approx 2{,}0$	✅
Удаление из середины	`list`	$\Theta(n)$	$T(2n)/T(n) \approx 2{,}5$	✅
Добавление в конец	Связный список	$\Theta(1)$	$T(2n)/T(n) \approx 2{,}0^{*}$	✅
Добавление в конец	`list`	$O(1)$ амортиз.	$T(2n)/T(n) \approx 1{,}8$	✅

* Рост $T(2n)/T(n) \approx 2$ для n добавлений в конец — это **линейный рост суммарного времени**, то есть одна операция $\Theta(1)$.

---

8. Выводы

1. Эксперимент **точно подтверждает** теоретические оценки. Доступ по индексу и удаление из середины в связном списке дают $T(2n)/T(n) \approx 2{,}0$ — это **линейный** рост $\Theta(n)$, как и предсказывает теория. Для `list` доступ по индексу — $\Theta(1)$ (время настолько мало, что таймер его почти не различает), а удаление из середины — $\Theta(n)$ с маленькой константой (сдвиг памяти в C).

2. **Связный список проигрывает массиву** в обеих исследованных операциях (доступ по индексу и удаление из середины) при случайном доступе. Причина — необходимость прохода от начала списка. Выигрыш связного списка ($\Theta(1)$ на перестановку ссылок) проявляется только тогда, когда **узел уже найден** — например, при удалении из очереди (`dequeue`), где всегда удаляется первый узел (`head`), или при отмене конкретной правки по сохранённой ссылке.

3. **Очередь на связном списке** реализует обе операции (`enqueue` и `dequeue`) за $\Theta(1)$ и не требует сдвигов, в отличие от очереди на массиве. Для потоковой обработки заявок (FIFO) это оптимальный выбор, если заранее неизвестен объём очереди. Однако из-за накладных расходов на узлы она занимает в 2,5–3 раза больше памяти, чем массив (Т4).

4. **Прикладная задача «Журнал операций с правками»**: связный список выигрывает только в **экзотическом** сценарии, когда все удаления идут по сохранённым ссылкам, а доступ по номеру не нужен. В реальном журнале, где регулярно выполняется отображение по номеру и отмена последней операции, **массив** (`list`) оказывается быстрее за счёт $\Theta(1)$ доступа по индексу и $\Theta(1)$ отмены последней операции. Оптимальная структура — **двусвязный список** (если нужна отмена по ссылке и доступ к соседним элементам) или **массив** (если преобладают доступ по номеру и добавление в конец).

5. **Расход памяти (Т4)**: связный список занимает ≈ 96 байт/элемент против ≈ 36 байт/элемент у `list` — разница в 2,5–3 раза. Для 10 000 элементов это ≈ 880 КБ, что при больших объёмах становится существенным. Это ещё один аргумент в пользу массива, когда данные помещаются в память.

6. **Общий вывод:** структуру данных следует выбирать по тому, **какие операции преобладают**. Для частого доступа по номеру и добавления в конец — массив. Для частого удаления из начала или из середины по известной ссылке — связный список. Для очереди — кольцевой буфер (массив с двумя индексами) или связный список с `head` / `tail`. Универсально лучшей структуры не существует.

---

9. Ответы на контрольные вопросы

1. **АТД** задаёт набор операций и их семантику, но не способ хранения. Пример: АТД «стек» с операциями `push`, `pop`, `top`. Две реализации: (а) на массиве — все операции $\Theta(1)$; (б) на связном списке — тоже все $\Theta(1)$, но с накладными расходами на узлы. Поведение (LIFO) одинаковое, а память и константы — разные.

2. **LIFO** (last in, first out) — последним пришёл, первым вышел. Примеры: отмена действия (undo), проверка парности скобок. **FIFO** (first in, first out) — первым пришёл, первым вышел. Примеры: очередь заявок службы поддержки, буфер ввода-вывода.

3. **Доступ по индексу**: в массиве адрес элемента вычисляется как `base + i * size` за $\Theta(1)$. В связном списке узлы расположены в памяти произвольно, и между ними только ссылки; чтобы найти i-й, нужно пройти i ссылок, то есть $\Theta(i)$.

4. **Амортизированная сложность** — средняя стоимость операции в серии из n операций. Вставка в конец динамического массива: при исчерпании ёмкости массив удваивается, и все элементы копируются — $\Theta(n)$ на одну операцию. Но удвоение происходит после $n/2$ вставок, поэтому суммарные затраты на n вставок — $\Theta(n)$, средняя — $O(1)$. Отсюда амортизированная стоимость $O(1)$.

5. **Ссылка на хвост** позволяет вставлять в конец за $\Theta(1)$ (без неё пришлось бы проходить весь список до последнего узла, $\Theta(n)$). Это стандартный приём для реализации очереди на связном списке.

6. **Двусвязный список** хранит ссылку `prev` в каждом узле, что позволяет удалять узел по ссылке за $\Theta(1)$ **без знания предшественника**, а также обходить список в обе стороны. Цена — дополнительная память на `prev` (ещё 8 байт на узел) и более сложные операции вставки/удаления (нужно обновлять два указателя).

7. **Кольцевой буфер** — массив фиксированной ёмкости m с двумя индексами: `head` (на извлечение) и `tail` (на вставку). При достижении конца массива индексы переходят в начало через операцию `mod m`: `tail = (tail + 1) % m`. Взятие остатка обеспечивает «заворачивание» индексов без сдвигов, давая $\Theta(1)$ на `enqueue` и `dequeue`.

8. **Массив часто быстрее связного списка на практике**, потому что:

   - узлы связного списка разбросаны по памяти, что плохо для кэша процессора (cache miss на каждом переходе);
   - каждый узел — отдельный объект Python с заголовком (~56 байт), что даёт накладные расходы;
   - массивы реализованы на C с оптимизированными операциями (сдвиг памяти через `memmove`).

   Асимптотика $\Theta(1)$ у массива против $\Theta(n)$ у списка справедлива, но на малых n константы могут «перевесить» асимптотику.

9. **Очередь через два стека**: `in_stack` — для `enqueue`, `out_stack` — для `dequeue`. При `enqueue(x)` кладём `x` в `in_stack`. При `dequeue()`: если `out_stack` пуст, перекладываем все элементы из `in_stack` в `out_stack` (порядок меняется на обратный), затем `pop()` из `out_stack`. Амортизированная стоимость каждой операции — $O(1)$: каждый элемент перекладывается не более одного раза.

10. **Журнал последних 100 событий** — это классический **кольцевой буфер** фиксированной ёмкости 100. При добавлении нового события, если буфер заполнен, самое старое событие автоматически «вытесняется» (заменяется новым). Это даёт $O(1)$ на добавление и $O(1)$ на доступ к любому из последних 100 событий. Альтернатива — `collections.deque(maxlen=100)`, который реализует ту же идею на C.

---

10. Рекомендуемая литература

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 10, 17.
2. Вирт Н. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: ДМК Пресс, 2016.
3. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 1.3.
4. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2017.
5. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.
6. Документация Python: модуль `collections`, класс `deque` [Электронный ресурс]. — URL: https://docs.python.org/3/library/collections.html